Handwritten Digit Classification System Powered by FPGA

Design Specification Document

IC Project-1: Group 7

Ashwin Karthik VK & Giridhar Ramesh

Lund University

1 Purpose

This document specifies the requirements and interface of `CNN_Top_small_PYNQ`, a hardware-folded convolutional neural network (CNN) inference accelerator implemented in RTL. The accelerator is designed for deployment on the PYNQ-Z1 and RFSOC 2x2 development boards and performs handwritten digit classification on 28×28 greyscale images (MNIST).

The primary objective of the design is to offload CNN inference onto the FPGA subsystem while fitting within the limited BRAM, DSP, and LUT budget of the device.

2 Architecture Overview

The accelerator is a single-clock, AXI4-Stream interfaced pipeline consisting of four cooperating sub-modules. An image is streamed in over the slave AXI4-S port, processed through three hardware stages (Convolution layer, Maxpool layer and Fully Connected layer) and the predicted digit is returned over the master AXI4-S port.

2.1 Sub-Module Summary

Module	Role	Description
`CNN_Top_small_PYNQ`	Top-level	Hosts the image buffer, row sequencer, fold accumulator FSM, requantisation, find max digit, and AXI4-Stream I/O.
`Conv_Filter_Bank_0_to_7`	Convolutional	8 channel folded 3×3 convolution filter. Shared FSM, BRAM accumulators, MAC datapath (3 DSPs), and int8 requantisation pipeline.
`MaxPool_Bank_0_to_7`	Pooling	8 channel 2×2 max-pooling bank. Processes two consecutive conv output rows per fold. Outputs 13-wide feature maps per channel.
`FC_Layer_Bank_0_to_7`	Fully Connected	8-channel FC layer. BRAM weight ROMs (per engine), two-DSP MAC per channel, 10-class int32 accumulator output per pass.

2.2 Data Flow

The end-to-end data path for a single inference is:

1. AXI4-S slave → 784-byte image buffer (BRAM, 28×28 pixel)
2. Row sequencer feeds 3 consecutive image rows (28 pixel each) to the Conv Bank per kernel-row triplet

3. `Conv_Filter_Bank_0_to_7` performs 8 parallel 3×3 int8 convolutions, producing int8 26-pixel wide feature maps

4. `MaxPool_Bank_0_to_7` applies 2×2 max-pooling to adjacent row pairs, giving 13-pixel wide output

5. `FC_Layer_Bank_0_to_7` multiplies 13-entry feature vectors by BRAM-resident weight matrices, accumulating 10 int32 class scores

6. The Fold Accumulator (in `CNN_Top`) repeats steps 2 to 5 for `fold_idx` 0→3, summing class scores across all 32 filters

7. FC biases are added; the 10 sums are requantised to int8 via 4 time-multiplexed DSP slices

8. Find highest sum over 10 int8 scores → 4-bit predicted digit → AXI4-S master output (1 byte)

3 Functional Requirements

Sl no	Priority	Requirement
1	Must	Accept a 28×28 greyscale image (784 bytes, uint8) over an AXI4-Stream slave interface.
2	Must	Assert `s_axis_tready` when the image buffer is not locked (ready to receive pixels).
3	Must	Latch the incoming pixel stream into a 784×8-bit BRAM image buffer; lock the buffer on `s_axis_tlast`.
4	Must	Perform a hardware-folded 2D convolution with 32 filters (3×3 kernel, int8 weights) producing a 26×26 feature map per filter.
5	Must	Time-multiplex all 32 filters across 4 fold passes (`fold_idx` 0–3), each pass processing 8 filters in parallel.
6	Must	Convert uint8 pixels to int8 by flipping the MSB (subtract 128) before convolution.
7	Must	Requantise convolution accumulator output to int8 using per-filter scale factors (`Q_FIXED`, `RSHIFT`) and zero-point (−128).
8	Must	Apply 2×2 max-pooling with stride 2 to each 26×26 feature map, producing a 13×13 output per filter.
9	Must	Compute a fully connected (FC) layer producing 10 int32 accumulators (one per digit class) per filter bank pass.
10	Must	Accumulate FC outputs across all 4 fold passes and add per-class biases (`FC_BIAS_0` ... `FC_BIAS_9`).
11	Must	Requantise the 10 summed FC scores to int8 (4 DSPs, time-multiplexed over 3 groups using `FC_Q` and `FC_R`).
12	Must	Perform argmax over the 10 int8 scores and output the predicted digit (0–9) as a 4-bit value.

Sl no	Priority	Requirement
13	Must	Transmit the predicted digit over an AXI4-Stream master interface as one byte, with `m_axis_tlast` asserted on the valid beat.
14	Must	Release the image buffer (allow next image) after the output handshake (`m_axis_tvalid` & `m_axis_tready`).
15	Must	Accept an active-low synchronous reset (`rst_n`) that returns all FSMs and output registers to a defined idle state.
16	Should	Support a `soft_clear` signal that flushes the Conv/MaxPool/FC pipeline FSMs mid-inference without full chip reset.
17	Should	Infer the 784-entry image buffer as a single-port BRAM (`ram_style` = block) to minimise LUT/FF resource usage.
18	Should	Infer 8×26-entry convolution accumulators as BRAMs to avoid exceeding BRAM port limits.

4 Interface

4.1 Port List

Signal	Direction	Width	Description
`clk`	Input	1	System clock. All registers are clocked on the rising edge.
`rst_n`	Input	1	Active-LOW asynchronous-capable reset. Drives FSM state, counters, and output registers to idle. `img_buf` write path has no reset.
`s_axis_tdata`	Input	8	Incoming pixel byte (uint8, 784 bytes per image).
`s_axis_tvalid`	Input	1	AXI4-S slave: producer indicates `s_axis_tdata` is valid.
`s_axis_tready`	Output	1	AXI4-S slave: asserted when the accelerator is ready to accept a pixel (`pix_cnt` $<$ 784 AND buffer not locked).
`s_axis_tlast`	Input	1	Marks the final pixel of an image frame (byte 783). The buffer is locked and `new_image_pulse` is generated on this beat.
`m_axis_tdata`	Output	8	Outgoing result byte [7:0]. Bits [7:4] = 0, bits [3:0] = predicted digit (0–9).
`m_axis_tvalid`	Output	1	AXI4-S master: asserted when `m_axis_tdata` holds a valid inference result.
`m_axis_tready`	Input	1	AXI4-S master: downstream consumer ready to accept the result.

Signal	Direction	Width	Description
m_axis_tlast	Output	1	Packet boundary signal. Tied to m_axis_tvalid (single-beat packet per inference; avoids constant-driver synthesis warning).

5 Internal FSM Summary

The design contains six FSMs spread across its four modules. All FSMs are clocked on the rising edge of clk and are reset by rst_n. Sub-module FSMs also respond to soft_clear (issued as new_image_pulse from CNN_Top at the start of each inference).

FSM	Module	States	Role
Row Sequencer	CNN_Top	SEQ_IDLE, SEQ_FETCH, SEQ_ASSERT, SEQ_KWAIT, SEQ_OWAIT, SEQ_DONE, SEQ_WAIT_FOLD	Feeds 28-pixel image rows to the conv bank one row at a time. SEQ_FETCH reads img_buf via single BRAM port (29 cycles).
Fold Accumulator	CNN_Top	FA_IDLE, FA_ACCUM, FA_WAIT_NEXT	Accumulates 8 FC-bank outputs per fold, manages fold_idx (0→3), issues seq_restart_req, asserts all_folds_done after fold 3.
Requantisation	CNN_Top	RQ_IDLE, RQ_C0, RQ_C1, RQ_C2, RQ_C3	Time-multiplexes 4 DSPs over digits 0–9 in 3 groups (4+4+2). Outputs score_int8[0:9] + rq_done.
Conv Bank Control	Conv_Filter	IDLE, FETCH, CONV_STEP, RQ_READ, RQ_BIAS, RQ_MULT, RQ_SHIFT	Single FSM drives all 8 parallel conv datapaths. Krow counter 0→2 gates BRAM overwrite vs. accumulate logic.
MaxPool Control	MaxPool_Bank	Implicit (row_parity + pool_active flags)	Two-row parity toggle. pool_active drives col_cnt 0→12 to emit 13 pooled outputs per valid pair.
FC Bank Control	FC_Layer	IDLE, BRAM_RD, COMPUTE, DRAIN, NEXT_PAIR, DONE	Sequences BRAM weight reads (2 DSPs per engine), pair_cnt (0→3), row_idx (0→12), and fold-extended addresses.

6 Constraints and Limitations

Sl no	Category	Constraint
1	**Target device**	Xilinx Zynq xc7z020clg400-1 on PYNQ-Z1 board.
2	**Input image size**	Fixed 28×28 greyscale (784 bytes). No other resolution is supported.
3	**Pixel format**	Unsigned 8-bit (uint8), row-major, MSB-first. Internally converted to int8 by MSB flip.
4	**Number of filters**	Exactly 32 convolutional filters (8 per fold pass × 4 passes). Not run-time configurable.
5	**Kernel size**	3×3 int8 weights only. Stride = 1; padding = none → 26×26 output feature map.
6	**Pooling**	2×2 max-pool with stride 2 (no padding) → 13×13 per-channel output.
7	**Output classes**	10 classes (digits 0–9). FC layer produces one int32 accumulator per class.
8	**Throughput model**	One inference per input image. The accelerator is not pipelined across images; next image is accepted after `m_axis` handshake completes.
9	**BRAM usage**	`img_buf` (784×8) must infer as a single RAMB36E1 block. Conv `acc_mem` (8×26×19) must also use BRAM. FC weight ROMs use block RAM.
10	**DSP usage**	FC requantisation uses exactly 4 DSP slices, time-multiplexed. Conv datapaths use LUT multipliers and 2 DSPs.
11	**Weight precision**	All weights int8 (−128 to +127). Biases int32. Q-factor and RSHIFT values are hardcoded constants.
12	**Reset polarity**	Active-LOW (`rst_n`). Must be connected to `peripheral_aresetn` on PYNQ-Z1.
13	**Timing budget**	Row sequencer fetch (29 cycles) + inter-krow gap (55 cycles) = 84 cycles per kernel-row assertion. Conv bank requires $\leq$ 52 cycles per krow — satisfied by design.